

SIMATS ENGINEERING

ASSESSMENT TOOL 1

Field	Details
Course Name	Internet Programming
Course Code	CSA43
Course Outcome	CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems (L4, L5)
Assessment Tool	Industry Problem Task (BL4–BL6)
Weightage	40%
Total Marks	30

Code of Conduct

I Adhil Bijukumar (192411133) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Adhil Bijukumar

Question 1 — SOAP-Based Web Service for Hospital Management System (10 Marks)

1.1 Problem Restatement

A hospital management system needs a SOAP-based web service to handle patient registration, appointment scheduling, and medical record management. Communication between hospital departments (Reception, Outpatient, Pharmacy, Billing, Laboratory) must be secure, structured, and interoperable, since departments may run on different platforms and technologies.

1.2 Structure of SOAP Messages

A SOAP message is an XML document wrapped inside an Envelope, and it is transport-independent (it can travel over HTTP, SMTP, or JMS). It has three parts:

- **Envelope:** The root element that identifies the XML document as a SOAP message and defines the namespace. It wraps the Header and Body.
- **Header (optional):** Carries meta-information such as authentication tokens, session IDs, transaction IDs, or WS-Security credentials. It does not carry the actual business data.
- **Body:** Contains the actual request or response data — the operation name and its parameters (e.g., patient ID, name, diagnosis) or a SOAP Fault if an error occurred.

Example — SOAP request to register a new patient:

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
  xmlns:hos="http://hospital.example.org/patient">
  <soap:Header>
    <hos:AuthToken>eyJhbGciOiJIUzI1NiJ9...</hos:AuthToken>
    <hos:DepartmentId>OPD-01</hos:DepartmentId>
  </soap:Header>
  <soap:Body>
    <hos:AddPatientRequest>
      <hos:Name>Ramesh Kumar</hos:Name>
      <hos:Age>34</hos:Age>
      <hos:Gender>Male</hos:Gender>
      <hos:ContactNumber>9876543210</hos:ContactNumber>
    </hos:AddPatientRequest>
  </soap:Body>
</soap:Envelope>
```

Example — corresponding SOAP response:

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
  xmlns:hos="http://hospital.example.org/patient">
  <soap:Body>
    <hos:AddPatientResponse>
```

```
<hos:PatientId>PT10245</hos:PatientId>
<hos:Status>Registered Successfully</hos:Status>
</hos:AddPatientResponse>
</soap:Body>
</soap:Envelope>
```

1.3 Operations Exposed by the Service

Operation	Purpose
addPatient(name, age, gender, contact)	Registers a new patient and returns a unique patient ID
scheduleAppointment(patientId, doctorId, date, time)	Books an appointment slot with a specified doctor/department
getPatientRecord(patientId)	Retrieves demographic details, visit history, and diagnosis
updateMedicalRecord(patientId, diagnosis, prescription)	Adds/updates clinical notes after a consultation
cancelAppointment(appointmentId)	Cancels a previously booked appointment

1.4 Client–Server Data Exchange Flow

- The client (e.g., the Reception module) constructs a SOAP request, wrapping method name and parameters in XML, and sends it as an HTTP POST to the service endpoint with the SOAPAction header set.
- The SOAP server receives the request, validates the envelope against the WSDL contract and XML Schema, and authenticates the caller using the header token.
- The server-side handler (e.g., a JAX-RPC/JAX-WS servlet) unmarshals the XML into Java objects, invokes the business logic (e.g., inserting the patient row into the hospital database), and marshals the result back into XML.
- The SOAP response is returned to the client over the same HTTP connection; the client unmarshals it and updates the UI or triggers the next workflow step (e.g., printing a patient ID card).
- If any step fails (e.g., invalid data), the server returns a SOAP Fault element inside the Body instead of a normal response.

1.5 Justification for Using SOAP

SOAP is the appropriate choice for this hospital scenario for the following reasons:

- Reliability: SOAP supports WS-ReliableMessaging and works well with transactional, ACID-compliant operations — critical when writing to a shared patient database where a lost or duplicated message (e.g., double-booking an appointment) is unacceptable.

- **Security:** SOAP integrates with the WS-Security specification, enabling XML encryption, digital signatures, and token-based authentication of the message itself — not just the transport channel — which matters for protecting sensitive medical data under regulations such as HIPAA.
- **Strict contracts:** Because SOAP services are described by a WSDL and validated against an XML Schema, every department's system knows exactly what data types and operations to expect, reducing integration errors between heterogeneous hospital systems.
- **Protocol independence:** SOAP can run over HTTP, SMTP, or JMS, giving the hospital flexibility if some departments use different transport infrastructure or message queues.

Question 2 — WSDL Design for an Enterprise Web Service (10 Marks)

2.1 Problem Restatement

An enterprise application must expose its web services so that external systems can discover and correctly invoke them. WSDL (Web Services Description Language) is used to describe the service's operations, message formats, and endpoint location in a machine-readable, XML-based contract.

2.2 Key Components of WSDL

Element	Role
types	Defines the data types used by the service, typically using XML Schema (XSD), e.g., the structure of a request or response object
message	Defines the abstract data being communicated — the input and output parameters of an operation, built from the types
portType (interface in WSDL 2.0)	Defines the set of operations (methods) the service offers and the messages each operation uses for input/output/fault
binding	Specifies the concrete protocol and data format (e.g., SOAP over HTTP) used to implement the portType's operations
service	Groups related ports together and specifies the network address (URL endpoint) where the service can be accessed

2.3 Sample WSDL Structure

Below is a simplified WSDL for a service exposing retrieveData and updateData operations:

```
<?xml version="1.0"?>
<definitions name="DataService"
  targetNamespace="http://enterprise.example.org/data"
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:tns="http://enterprise.example.org/data">

  <types>
```

```

<xsd:schema targetNamespace="http://enterprise.example.org/data">
  <xsd:element name="DataRequest">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="recordId" type="xsd:string"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
  <xsd:element name="DataResponse">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="recordValue" type="xsd:string"/>
        <xsd:element name="status" type="xsd:string"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
</types>

<message name="RetrieveDataRequest">
  <part name="parameters" element="tns:DataRequest"/>
</message>
<message name="RetrieveDataResponse">
  <part name="parameters" element="tns:DataResponse"/>
</message>

<portType name="DataServicePortType">
  <operation name="retrieveData">
    <input message="tns:RetrieveDataRequest"/>
    <output message="tns:RetrieveDataResponse"/>
  </operation>
  <operation name="updateData">
    <input message="tns:RetrieveDataRequest"/>
    <output message="tns:RetrieveDataResponse"/>
  </operation>
</portType>

<binding name="DataServiceBinding" type="tns:DataServicePortType">
  <soap:binding transport="http://schemas.xmlsoap.org/soap/http"
    style="document"/>
  <operation name="retrieveData">
    <soap:operation soapAction="http://enterprise.example.org/retrieveData"/>
    <input><soap:body use="literal"/></input>
    <output><soap:body use="literal"/></output>
  </operation>
</binding>

<service name="DataService">
  <port name="DataServicePort" binding="tns:DataServiceBinding">
    <soap:address location="http://enterprise.example.org/services/data"/>
  </port>
</service>
</definitions>

```

2.4 How the Components Define the Service Interface

- types establishes the vocabulary — every request and response element is strongly typed using XSD, preventing malformed or mismatched data from being sent.
- message builds abstract, reusable data units from those types, which are then referenced by name inside portType operations.
- portType is the contract of what the service can do (its 'interface'), independent of how it is transported — this is what a consumer reads first to know the available operations.
- binding attaches a concrete protocol (SOAP/HTTP, document/literal style) to that abstract interface, specifying exactly how messages are serialized on the wire.
- service ties the binding to a real, invocable network address, so a client tool can generate a working proxy/stub directly from the WSDL.

2.5 Importance in Service Integration

- Language and platform independence: A Java-based enterprise service can be consumed by a .NET, Python, or PHP client because WSDL is a neutral, machine-readable contract.
- Automatic client generation: Tools (e.g., wsimport, wsdl2java) can generate client stubs directly from the WSDL, reducing manual integration errors.
- Contract-first development: Teams can agree on the WSDL/XSD contract before implementation, enabling parallel development of client and server.
- Discoverability: When published (e.g., via UDDI or an API catalog), WSDL lets external systems discover what operations exist and how to call them without reading source code.

Question 3 — Debugging a Faulty SOAP Communication System (10 Marks)

3.1 Problem Restatement

A company's SOAP-based system is experiencing incorrectly processed requests and delayed or incorrect responses. The task is to analyze the SOAP communication process, identify likely root causes, and propose a systematic debugging approach.

3.2 Possible Root Causes

Issue Category	Typical Symptom
Message formatting errors (malformed XML, missing closing tags, wrong element order for document/literal style)	Server rejects the request with a Client-side SOAP Fault or an XML parsing exception before business logic even runs

Incorrect or mismatched XML namespaces	Elements are not recognized by the server (deserialization fails silently or returns null values), even though the XML 'looks' correct
Endpoint / SOAPAction problems (wrong URL, missing or incorrect SOAPAction header, HTTP vs HTTPS mismatch)	HTTP 404/405 errors, connection timeouts, or the wrong operation being invoked
WSDL/schema mismatch between client and server versions	Requests that were valid against an old WSDL fail validation after the service contract changes
Network / infrastructure issues (firewall, load balancer timeout, DNS)	Intermittent delays or responses that arrive after the client has already timed out
Server-side processing errors (business logic exception, database timeout)	Server-side SOAP Fault with a generic or unclear fault string

3.3 Proposed Debugging Approach

- Capture raw traffic: Use a SOAP testing tool (e.g., SoapUI, Postman) or an HTTP logging proxy to capture the exact request and response XML exchanged, rather than relying only on application-level logs.
- Validate against the schema/WSDL: Run the captured request XML through an XSD/WSDL validator to confirm it conforms to the expected types, element order, and namespaces defined in the contract.
- Check namespace declarations: Compare the namespace URIs and prefixes used in the client message against those declared in the WSDL's targetNamespace; a mismatch here is one of the most common silent-failure causes.
- Verify the endpoint configuration: Confirm the client is pointing at the correct <soap:address> URL from the current WSDL, and that the SOAPAction header matches the operation being invoked.
- Inspect SOAP Faults in detail: Read the faultcode, faultstring, and faultactor/detail elements returned by the server, since these usually indicate exactly which stage of processing failed.
- Add structured server-side error handling: Wrap business logic in try/catch blocks and translate exceptions into meaningful, custom SOAP Fault detail elements instead of letting generic server errors propagate.
- Introduce logging and timeouts: Add request/response logging with timestamps and correlation IDs at each hop, and set explicit client-side timeout and retry policies to distinguish network delay from processing delay.

- Regression test after each fix: Re-run a suite of valid and intentionally invalid requests (boundary/negative testing) to confirm both correct requests succeed and malformed ones fail gracefully with a clear fault.

3.4 Example of a Well-Formed SOAP Fault

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Body>
    <soap:Fault>
      <soap:Code>
        <soap:Value>soap:Sender</soap:Value>
      </soap:Code>
      <soap:Reason>
        <soap:Text xml:lang="en">Invalid patientId format</soap:Text>
      </soap:Reason>
      <soap:Detail>
        <errorCode>ERR_INVALID_ID</errorCode>
      </soap:Detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

3.5 How the Corrected System Ensures Reliable Communication

- Contract-validated messages (checked against WSDL/XSD before processing) eliminate malformed-message failures at the boundary, rather than deep inside business logic.
- Consistent, well-declared namespaces ensure the server can always correctly deserialize incoming elements into the right objects.
- Explicit, descriptive SOAP Faults let calling systems programmatically detect and handle specific error conditions instead of generic failures.
- Correlation IDs and structured logging make it possible to trace a single transaction end-to-end across client, network, and server, isolating whether a delay is due to the network, the server, or the client.
- Defined timeout and retry policies, combined with idempotent operation design, prevent duplicate processing when a response is delayed rather than lost.